

Green-Finance

A Real-Time Energy Observability Framework for Sustainable Financial Microservices

Group: 13

Faculty : Prof. Kirtimalini Chaudhari

Team member:

Atharva Hule (2024800038)

Laukik Deshpande (2024800023)

Soham gaikwad (202480030)

Phase 2 Deliverables

This presentation covers the four Phase 2 evaluation components, building on the Phase 1 foundation.

P1 — SRS

Software Requirements Specification — completed and reviewed

P2 — UML Diagrams

Corrected UML models representing system design

P3 — Prototype

functional prototype screens

P4 — Presentation

Technical evaluation presentation (this deck)

SRS Overview

[link](#)

Document Purpose

The SRS defines all functional and non-functional requirements for the Green-Finance observability platform, serving as the authoritative reference for design, implementation, and validation.

Key Requirement Categories

- **Functional:** Real-time energy data ingestion, carbon footprint calculation, financial impact reporting
- **Non-Functional:** Low-latency processing, scalability, data security, API availability
- **System Interfaces:** REST APIs, microservice contracts, data schemas
- **Constraints:** Compliance, deployment environment, integration boundaries

UML Design Overview

The Green-Finance system design has been translated into a set of corrected UML diagrams. Each diagram captures a distinct aspect of system behavior, structure, or interaction — forming a complete and verifiable design model.

Use Case Diagram

Actor interactions and system functionality scope

Class Diagram

Static structure, entities, and relationships

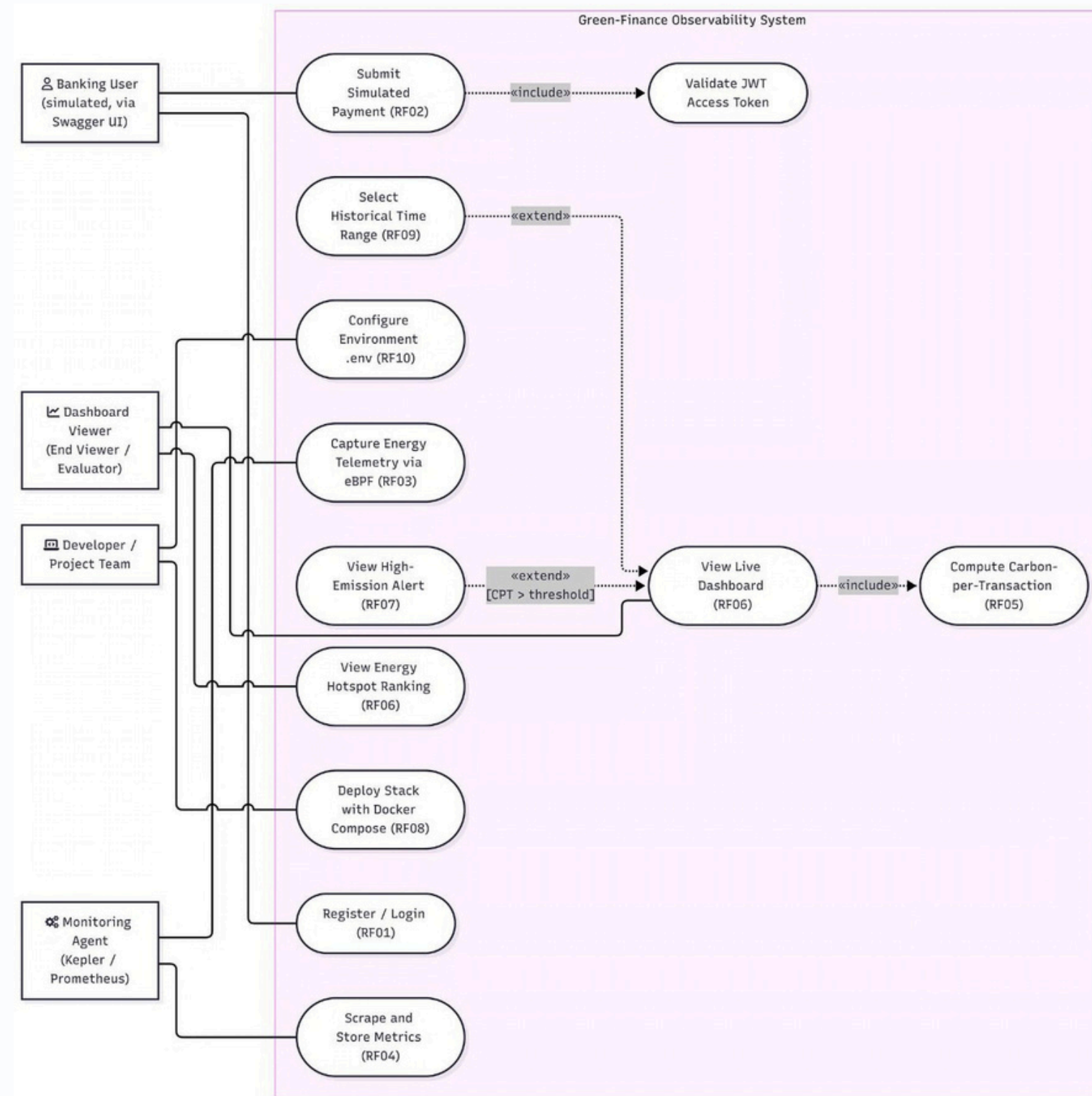
Sequence Diagram

Dynamic message flows between components

Activity Diagram

Operational workflow and decision logic

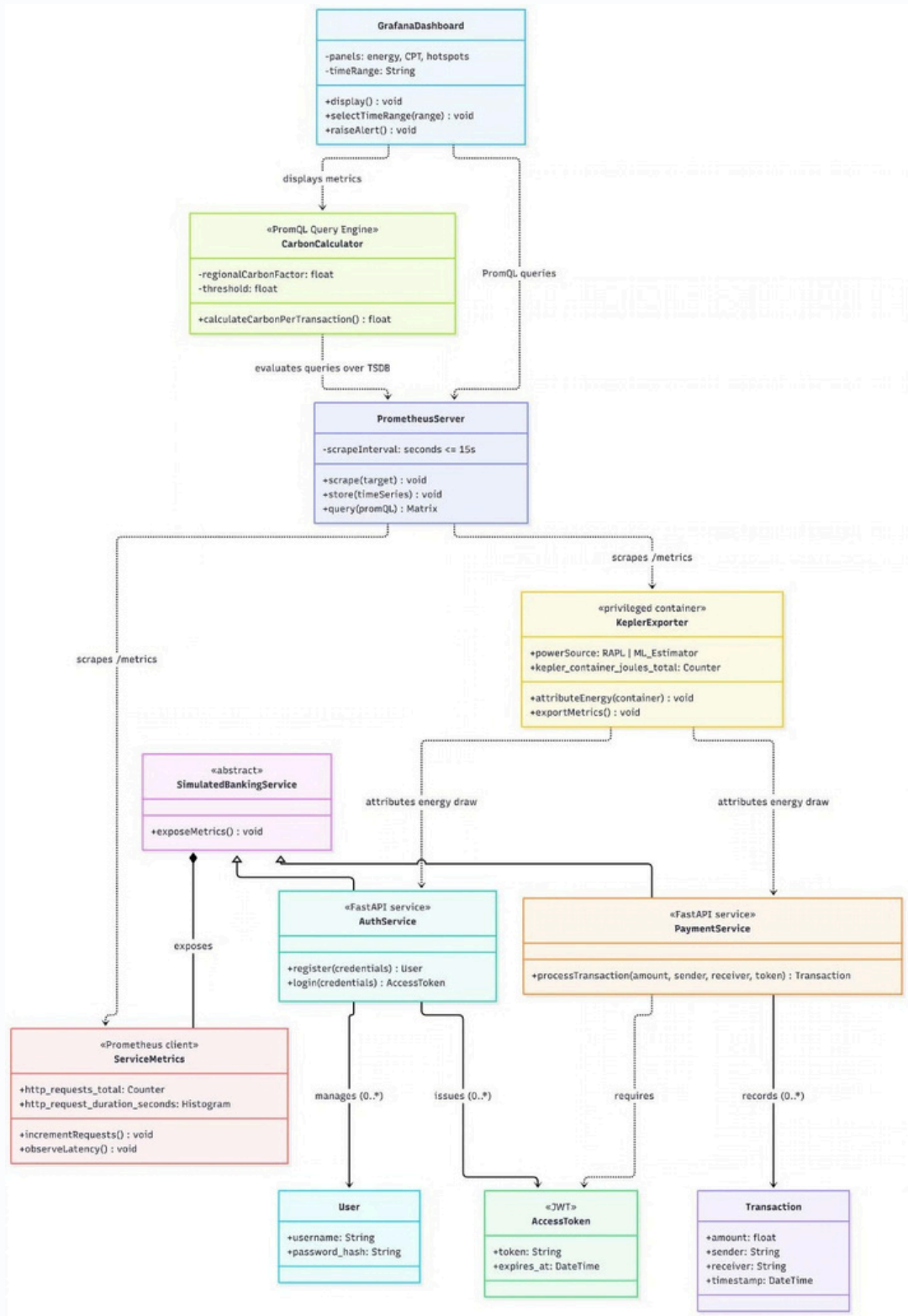
Use Case Diagram



What This Diagram Shows

- **Primary Actors:** Banking User (Swagger UI), Dashboard Viewer (Evaluator / End User), Developer / DevOps, and Monitoring Agent (Kepler & Prometheus).
- **Core Functional Scope:** Covers requirements **RF01 to RF10**, including simulated authentication, payment execution, eBPF energy capture, and Grafana visualization.
- **UML Relationships:**
 - «include» dependency between Payment Submission and JWT Access Token validation.
 - «extend» relationships for optional actions: Threshold Alerting (RF07) and Historical Range Queries (RF09) extending the Base Dashboard.
- **System Boundary:** Encapsulates API workloads, kernel telemetry probes, PromQL calculation, and Docker Compose orchestration.

Class Diagram



What This Diagram Shows

- **Four-Layer Architecture:** Organizes components into Simulated Banking, Energy Telemetry, Metrics Aggregation, and Visualization layers.
- **Domain Entities & Services:**
 - AuthService and PaymentService inheriting from abstract base SimulatedBankingService.
 - Core entities: User, AccessToken (JWT), Transaction, and Prometheus-instrumented ServiceMetrics.
- **Telemetry & Derived Computation:**
 - KeplerExporter attributes eBPF/RAPL joule counters to container targets.
 - CarbonCalculator combines time-series data with regional emission factors via PromQL to compute {gCO2} per transaction.
- **Multiplicity & Visibilities:** Explicit method signatures, attribute types, and scrape dependencies (<15s interval).

Sequence Diagram

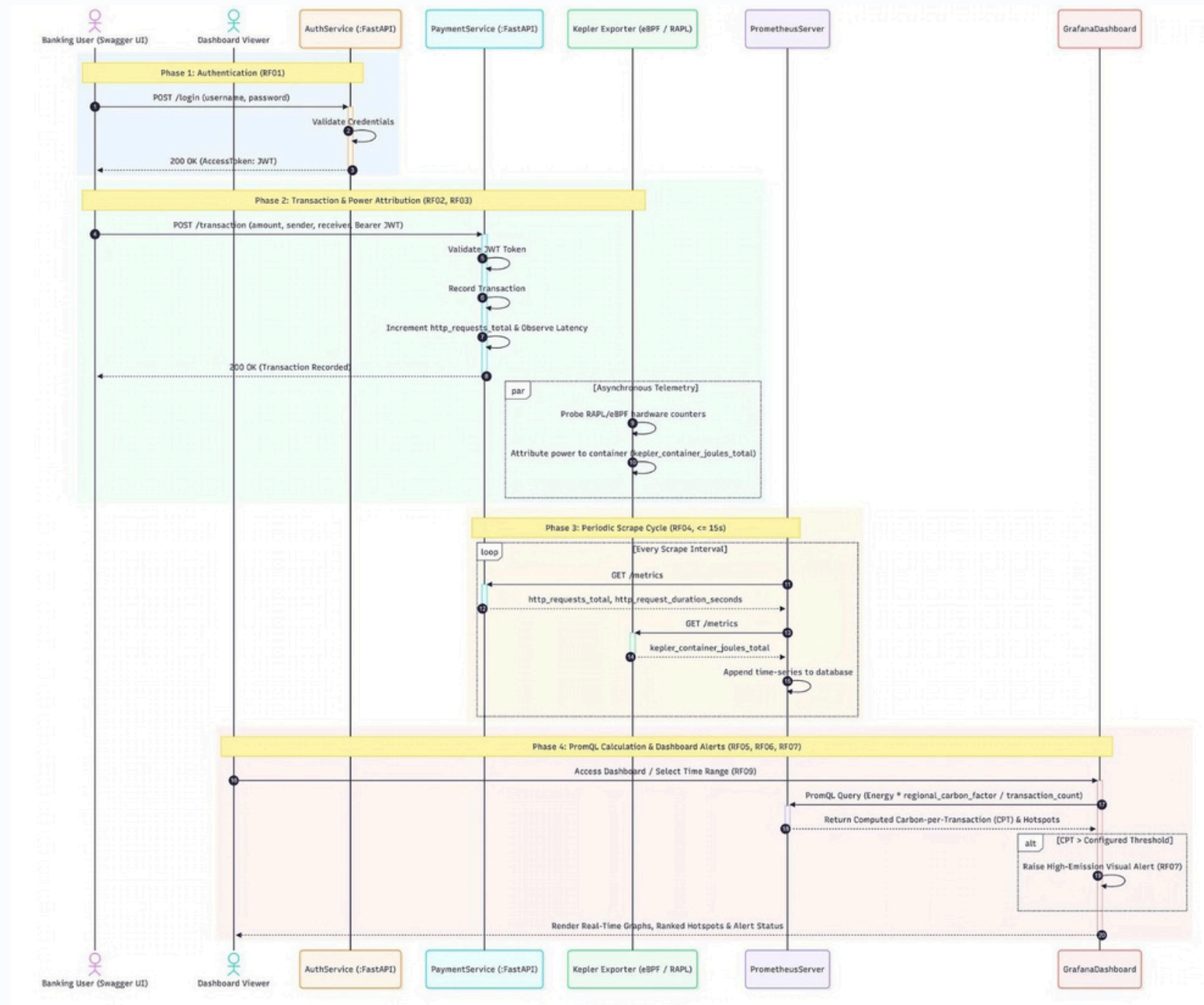
What This Diagram Shows

- **Chronological Message Flow:** Traces a request from user submission to real-time Grafana dashboard updates.

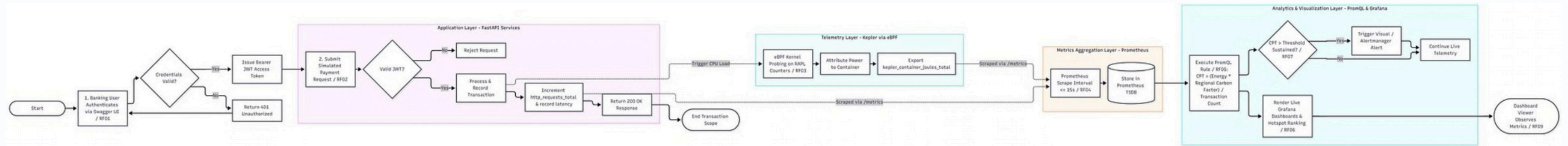
- **4 Distinct Execution Phases:**

1. *Authentication (RF01):* User login and JWT issuance.
2. *Payment & Telemetry (RF02, RF03):* Authenticated transaction write running in parallel with asynchronous eBPF kernel energy attribution.
3. *Periodic Scrape Cycle (RF04):* Prometheus scraping /metrics endpoints every <15s.
4. *PromQL & Visualization (RF05–RF07):* Grafana querying Prometheus, computing Carbon-per-Transaction, and evaluating alert thresholds.

- **Call Lifelines & Synchronicity:** Shows synchronous REST API calls alongside asynchronous hardware counter reading and scheduled pull-based scraping.



Activity Diagram



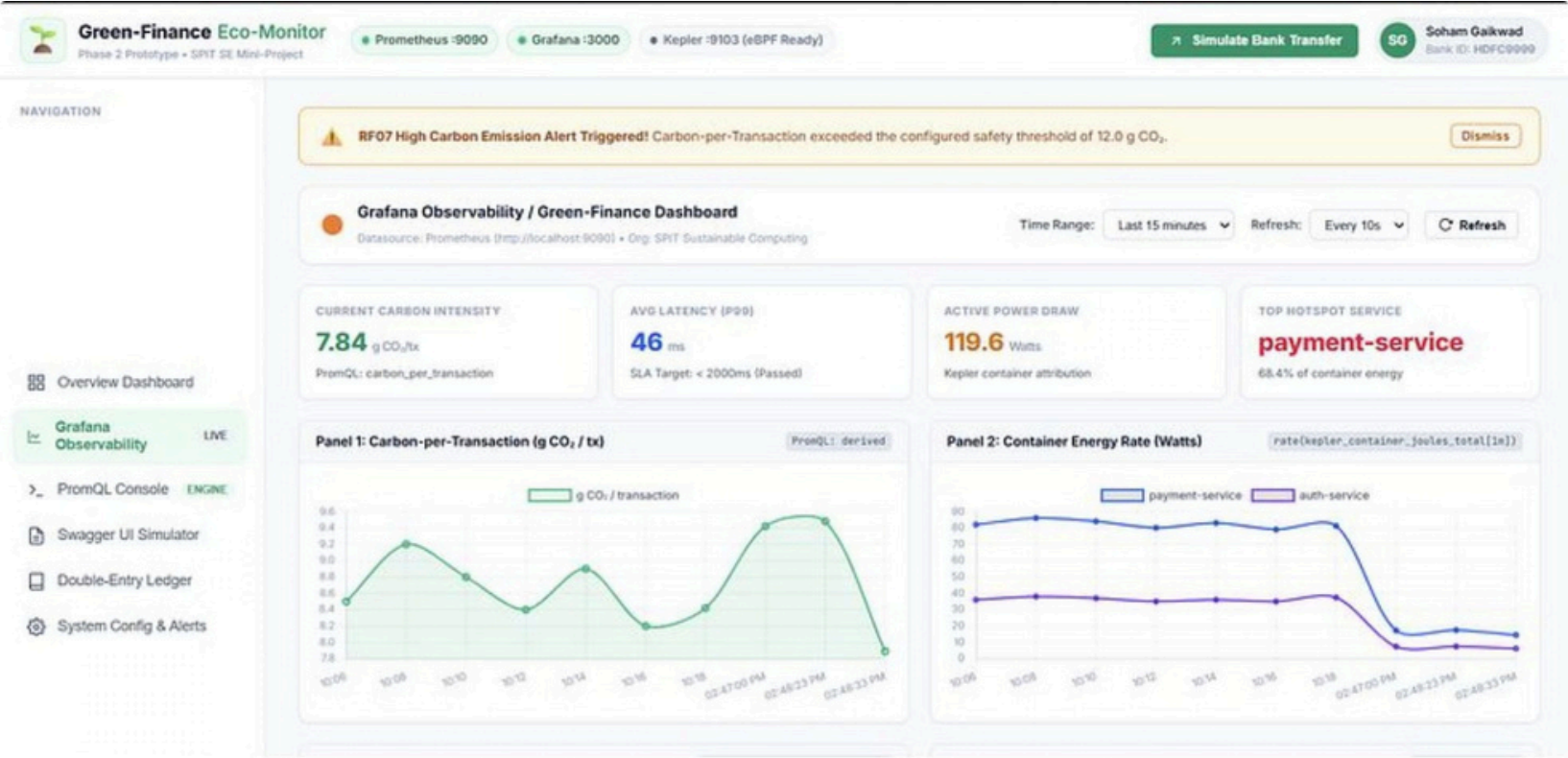
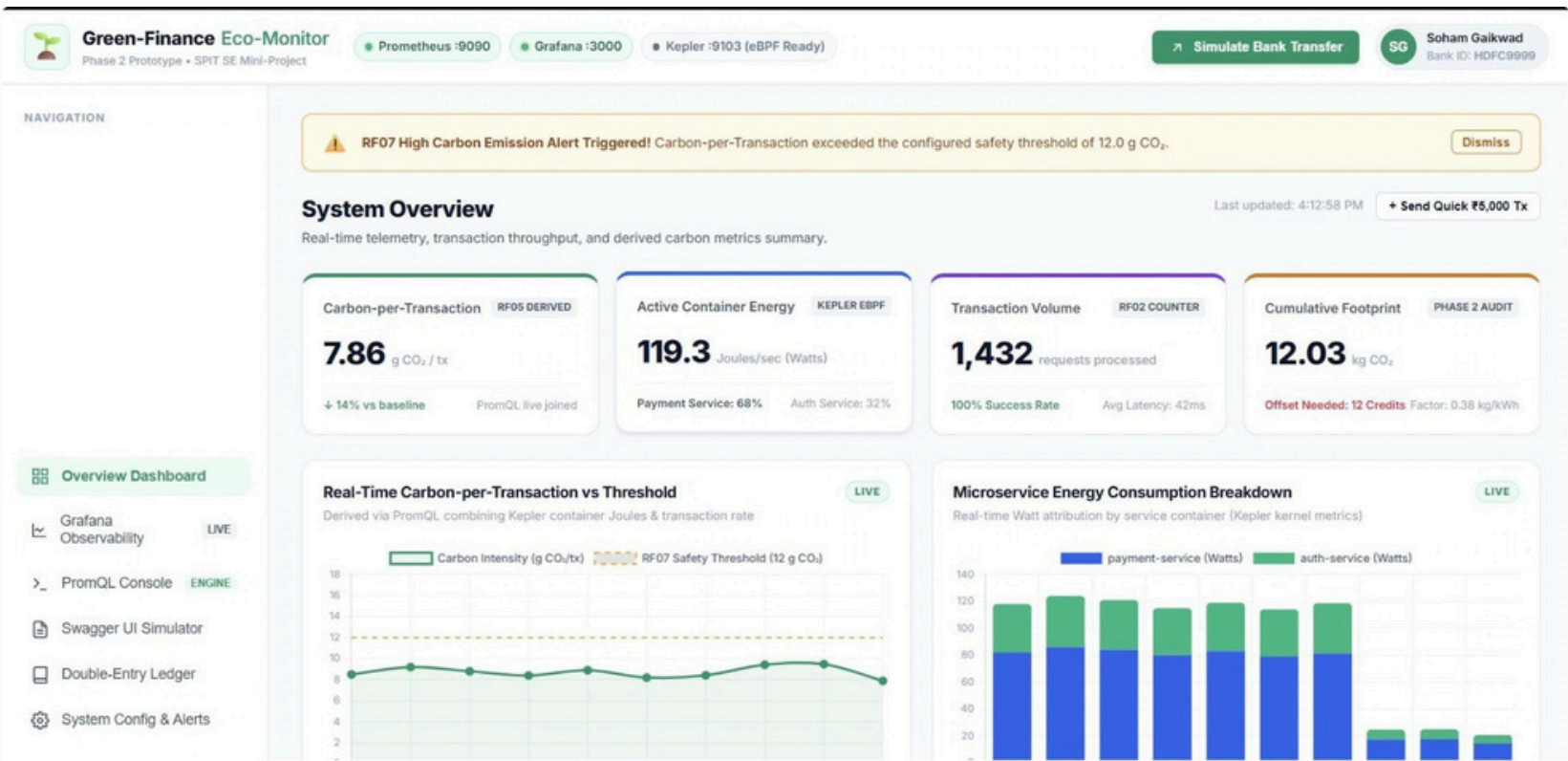
What This Diagram Shows

- **End-to-End Operational Pipeline:** Visualizes the complete workflow from token generation to sustainability dashboard rendering.
- **Parallel Workload Execution:** Fork/Join branches showing that application transaction logging occurs concurrently with kernel-level power attribution.
- **Multi-Source Scraping & Derivation:** Shows Prometheus aggregating application metrics (`http_requests_total`) and container power draw (`kepler_container_joules_total`) to compute derived CPT.
- **Decision Logic & Alert Handling:** Explicit conditional branches for invalid JWT rejection, authentication retries, and high-emission threshold breach triggers (RF07).

Prototype Demonstration

[github](#)

The functional prototype validates core user flows and UI interactions as specified in the SRS. Screens below represent key features of the Green-Finance interface.



Phase 2 Summary

SRS Completed

Full requirements specification documented, reviewed, and approved

UML Models Completed

Use Case, Class, Sequence, and Activity diagrams corrected and finalized

Prototype Developed

Functional prototype screens demonstrate core system features